

LAB4 REPORT

Name: Mourya Nandan

Roll No: CS23B006

QUESTION 1:

What we want to do:

- We want to implement two new system calls in xv6:
- `sigalarm(int ticks, void (*handler)());`
- `sigreturn(void);`
- **The purpose:**
 - `sigalarm(n, fn)` lets a process request that after every **n CPU ticks** it consumes, the kernel will interrupt the process and force execution of function `fn` in user space.
 - After `fn` finishes, it must call `sigreturn()`, which restores the process's registers and program counter so it continues from where it left off.
- This is like a **user-level timer interrupt/handler**

What I have done:

1) User space changes

- **Makefile (UPROGS section)**
 - Added `_alarmtest` so that `alarmtest.c` is compiled and available as a user program.
- **user/user.h**
 - Added prototypes:
 - `int sigalarm(int ticks, void (*handler)());`
 - `int sigreturn(void);`
- **user/usys.pl**
 - Added entries:
 - `entry(sigalarm)`
 - `entry(sigreturn)`
- This generates syscall stubs so user programs can call `sigalarm` and `sigreturn` like normal functions.
- `alarmtest.c` (provided test program)
 - Calls `sigalarm(2, periodic)` and then spins. The kernel should periodically jump to the handler `periodic()` which prints "alarm!".

2) Kernel space changes

- **kernel/syscall.**
 - Added syscall numbers:
 - `#define SYS_sigalarm 22`
 - `#define SYS_sigreturn 23`
- **kernel/syscall.c**

- Declared:
 - extern uint64 sys_sigalarm(void);
 - extern uint64 sys_sigreturn(void);
- Added to syscall table:
 - [SYS_sigalarm] sys_sigalarm,
 - [SYS_sigreturn] sys_sigreturn,
- **kernel/proc.h (struct proc additions)**
 - int alarm_interval;
 - int alarm_ticks_left;
 - uint64 alarm_handler;
 - int alarm_handling;
 - struct trapframe alarm_tf;
- **kernel/proc.c (allocproc)**
 - Initialized:
 - p->alarm_interval = 0;
 - p->alarm_ticks_left = 0;
 - p->alarm_handler = 0;
 - p->alarm_handling = 0;
- **kernel/sysproc.c**
 - Implemented:
 - uint64 sys_sigalarm(void) {
 - int ticks;
 - uint64 handler;
 - argint(0, &ticks);
 - argaddr(1, &handler);
 - struct proc *p = myproc();
 - acquire(&p->lock);
 - p->alarm_interval = ticks;
 - p->alarm_ticks_left = ticks;
 - p->alarm_handler = handler;
 - release(&p->lock);
 - return 0;
 - }
 -
 - uint64 sys_sigreturn(void) {
 - struct proc *p = myproc();
 - acquire(&p->lock);
 - *p->trapframe = p->alarm_tf;
 - p->alarm_handling = 0;
 - release(&p->lock);
 - return 0;
 - }
- **kernel/trap.c (usertrap)**
 - Inside timer interrupt section:
 - if(which_dev == 2) {
 - struct proc *p = myproc();

- if(p && p->alarm_interval > 0 && !p->alarm_handling) {
 - p->alarm_ticks_left--;
 - if(p->alarm_ticks_left == 0) {
 - p->alarm_tf = *p->trapframe;
 - p->alarm_handling = 1;
 - p->alarm_ticks_left = p->alarm_interval;
 - p->trapframe->epc = p->alarm_handler;
 - }
- }
- }

3) Execution Flow:

Setup:

- alarmtest calls sigalarm(2, periodic)
- Kernel sets interval = 2, ticks_left = 2, handler = address of periodic.

Timer interrupts:

- Each tick, ticks_left--.
- When it hits 0:
 - Kernel saves current trapframe (all registers + program counter).
 - Marks alarm_handling = 1.
 - Resets ticks_left = interval.
 - Replaces trapframe->epc with handler address.

Handler execution:

- User-space executes periodic() (prints "alarm!").
- periodic() ends by calling sigreturn().

sigreturn syscall:

- Restores trapframe from saved copy.
- Clears alarm_handling.
- Process resumes exactly where it was before the handler.

4) Question and Answers:

- **Why do we save the trapframe?**
 - Because when the handler runs, registers and program counter change. Without saving, we cannot return to the interrupted code correctly.
- **Why must the handler call sigreturn()?**
 - Because only the kernel knows how to restore the original trapframe. If handler returns normally, it won't go back to the right instruction.
- **What happens if sigalarm(0,0) is called?**

- It disables alarms: kernel never sets up handlers again.
- **Why do we use `acquire(&p->lock)` and `release(&p->lock)`?**
 - To prevent race conditions between syscalls (`sigalarm`) and timer interrupts (`usertrap`) that both access alarm fields. Without locking, fields like `alarm_ticks_left` could be corrupted.
- **Why not allow nested handlers?**
 - We prevent re-entry using `alarm_handling` flag. Otherwise, a timer firing inside an already-running handler would overwrite saved state and corrupt execution.
- **Why does `alarmtest` show multiple "alarm!"?**
 - Because every 2 ticks the handler is triggered, and it prints "alarm!".
- **What ensures the process resumes correctly after handler?**
 - `sigreturn()` restores the exact saved trapframe (registers, program counter). So execution continues as if the interrupt never happened.

OUTPUT:

```
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
...alarm!
...alarm!
...alarm!
...alarm!
..alarm!
..alarm!
...alarm!
..alarm!
..alarm!
...alarm!
test1 passed
test2 start
.....alarm!
test2 passed
test3 start
test3 passed
```